

aio-agent v2.1 — Resumen Ejecutivo

Propósito

aio-agent es un agente ligero de SRE que utiliza **function calling nativo** sobre el modelo local **Qwen2.5-7B-Instruct** (served by llama.cpp). Responde en español, ejecuta comandos Linux, lee archivos de configuración y logs, y escribe archivos bajo ciertas rutas permitidas.

Modelo definitivo

- **Qwen2.5-7B-Instruct-Q4_K_M** en `http://localhost:8083/v1/chat/completions`, con ventana de contexto de 8K y `MAX_HISTORY_TOKENS=6000`.
- Se evaluó **Qwen2.5-Coder-3B** y se descartó: su function calling era inconsistente (tool calls mal formados, funciones inventadas, ignoraba el esquema JSON).
- Qwen2.5-7B-Instruct emite `tool_calls` válidos de forma consistente y completa planes multi-paso.
- Los modelos antiguos de la familia Qwen3 (Qwen3-8B Q4_K_M/Q6_K/Q3_K_M y Qwen3-4B) ya han sido eliminados del servidor.

Cuantización del modelo

Cuantización	Velocidad (tok/s)	RAM/VRAM	Calidad observada	Caso de uso
Q4_K_M	57 prompt / 20 gen	4.7 GB	Referencia	Servidor productivo con suficiente memoria

La instancia desplegada actualmente usa **Qwen2.5-7B-Instruct-Q4_K_M**. Para evitar desbordamientos silenciosos de contexto, `agent.py` cuenta tokens con el endpoint real `/v1/tokenize` antes de comprimir o truncar el historial, en lugar de estimar con una relación fija de caracteres por token.

Funcionalidades v2.1

Funcionalidad	Estado
Function calling nativo vía llama.cpp	[OK]
<code>run_command</code> con timeout y captura de salida	[OK]
<code>read_file</code> con comprobaciones de permisos y tamaño	[OK]
<code>write_file</code> bloqueando rutas críticas del sistema	[OK]
Bucle conversacional con hasta 5 turnos tool→LLM	[OK]
Planificación y ejecución multi-paso sin intervención	[OK]
Seguridad básica: advertencia antes de comandos destructivos	[OK]
CLI interactiva en español	[OK]
Historial readline (flechas arriba/abajo, cursor)	[OK]
Memoria de sesión persistente	[OK]
README.md y PDF ejecutivo actualizados	[OK]

Arquitectura

`chat.py` -- `agent.py` -- `tools.py` -- Qwen2.5-7B-Instruct vía llama.cpp:8083

- `chat.py`: bucle interactivo con readline.
- `agent.py`: orquestador de function calling y memoria procedural.
- `tools.py`: definición y manejadores de herramientas.

Historial readline

La CLI ahora usa el módulo `readline` de Python:

- Flechas arriba/abajo para recorrer comandos anteriores.
- Flechas izquierda/derecha para mover el cursor.
- Historial persistente en `data/.chat_history` (hasta 500 entradas).

Uso

```
python3 chat.py
```

Ejemplos:

```
> muestra el uso de disco
> lee /var/log/syslog
> escribe un script de backup en /tmp/backup.sh
> reinicia nginx
```

Escribe `salir`, `exit` o `quit` para terminar.

Seguridad

- Advertencia antes de comandos destructivos.
- `write_file` bloquea `/etc`, `/boot`, `/sys`, `/proc`, `/dev`.
- La memoria conversacional se guarda en sesión; el historial readline solo guarda líneas de entrada del usuario.

Siguientes pasos recomendados

- Evaluar en escenarios reales de SRE (reinicio de servicios, diagnóstico de logs, backups).
- Añadir confirmación explícita del usuario para comandos destructivos.
- Integrar más herramientas: `git_operation`, gestión de paquetes, consulta de estado de `systemd`.

Documento generado el 22 de julio de 2026.